

Dynamont: Segmentation of the ONT signal with Dynamic Programming and HMMs

Jannes Spangenberg¹, Christian Höner zu Siederdissen¹, and Manja Marz^{1,2,3}

¹*RNA Bioinformatics and High-Throughput Analysis, Friedrich Schiller University Jena, Leutragraben 1, 07743 Jena, Germany*

²*European Virus Bioinformatics Center 2, Leutragraben 1, 07743 Jena, Germany*

³*FLI Leibniz Institute for Age Research, Beutenbergstraße 11, 07745 Jena, Germany*

January 29, 2025

lennart als co-author

Abstract

TODO

Introduction

* Oxford Nanopore Technologies (ONT) sequencing has revolutionized the field of deoxyribonucleic acid (DNA) and ribonucleic acid (RNA) sequencing [1–3]. It offers a unique approach to sequencing, allowing to directly sequence long native DNAs or RNAs, while keeping nucleotide modifications intact and detectable [4, 5]. ONT sequencing works by passing a DNA or RNA molecule through a biological pore in a membrane, while an electrical field is applied to it [6, 7]. As the molecules pass through the pore, changes in the electrical current (pico Ampere (pA)) are measurable. The resulting ONT signal is a pA time series. Basecallers convert the signal back into the nucleotide sequence [8, 9].

Tools like **Tombo** from ONT and **f5c** [10] (an optimized re-implementation of **nanopolish** [11]) produce signal to read alignments. The assignment of nucleotide to signal segment enables deeper downstream analysis of the data (e.g. nucleotide modifications). However, these tools often face challenges with modifications because the signal distribution models they rely on (k -mer models) do not account for nucleotide modifications. Additionally, proper comparisons and evaluations of the tools’ alignment capabilities are lacking due to the absence of a standardized gold-standard dataset.

In this paper, we introduce **Dynamont**, a tool that utilizes dynamic programming of an hidden Markov model (HMM) specifically designed for analyzing the ONT sequencing signal. **Dynamont** aligns reads to the corresponding ONT signals. This information is crucial for further signal processing and downstream analysis and allows researchers to gain insights into the relationship between

the read and the ONT signal, ultimately leading to a better understanding of the sequencing results.

*

*

- how much better is dynamont vs the other tools?
- list key results here
- mention the possibility for rna modifications here!

*

Materials & Methods

Datasets

HEK293T

* We downloaded the HEK293T wild type replicate 1 that is available through the ENA (PR-JEB40872). The dataset contains *in vivo* direct RNA sequencing (DRS) reads, sequenced on a MinION with a r9.4.1 flow cell [12]. In this study, we used 64,000 randomly selected reads from the dataset.

put readids.txt somewhere

We base called them using **Dorado** (v0.8.3) using the **rna002_70bps_hac@v3** model with default parameters, Sec. S1.1. We downloaded the complementary DNA (cDNA) and toplevel DNA from the ensemble database (release 112) to use as reference sequences for this dataset. *

rp4 UHR

* To benchmark the tools on the new RNA pore 4 (RP4), we downloaded the universal human reference RNA benchmark dataset from the Garvan Institute Long Read Sequencing Benchmark Data. This dataset also contains *in vivo* DRS reads, sequenced on a PromethION with a RP4 flow cell, available via the <https://gtgseq.s3.amazonaws.com/index.html#ont-rna004-rna/UHR/raw/amazon> web service. We extracted 10,000 randomly selected reads from the dataset to use in this study.

put readids in supplements?

Using **Dorado** (v0.8.3) with default parameters and the **rna004_130bps_sup@v5.1.0** model we base called the reads, Sec. S1.2. *

general definitions

* The ONT signal is a time series, denoted by T , which represents the pA measurements:

$$T = (t_0, \dots, t_{\mathcal{T}-1}) \quad \text{of length } \mathcal{T}$$

The read corresponds to a nucleotide sequence, denoted by N , where each element n_j is a nucleotide:

$$N = (n_0, \dots, n_{\mathcal{N}-1}) \quad \text{of length } \mathcal{N}, \quad \text{where } \forall j : n_j \in \Sigma$$

The set of possible nucleotides in N is denoted by Σ :

$$\Sigma = \{A, C, G, T\}$$

The set C represents all possible k -mers and is given by:

$$C = \Sigma^k \quad \text{with} \quad |C| = |\Sigma|^k$$

A k -mer is a sequence of length k formed from the elements of Σ and is therefore an element of C :

$$k\text{-mer} \in C$$

Dynamont iterates over T and N , aligning t_i with k -mer k_j of the sequence N :

$$t_i \quad \text{for} \quad 0 \leq i < \mathcal{T} \quad \text{and} \quad k_j = (n_j, \dots, n_{j+k-1}) \quad \text{for} \quad 0 \leq j < \mathcal{N} - k$$

$$\mathcal{N} - k + 1 \quad k\text{-mers exist in} \quad N$$

k depends on the pore type used for sequencing, e.g. for sequencing RNA with the r9 pore: $k = 5$, in case of the rp4 pore: $k = 9$. *

Data preprocessing

*

Normalization

All ONT signals are provided in **fast5** or **pod5** (from ONT), or in **slow5** [13] format. Signals are stored as integers values T_{DACs} to reduce memory space, they can be converted to the pA signal T_{pA} using Equ. 1. The offset, range, and digitization parameters are stored next to each read in the raw data. Each signal is normalized, Equ. 2 and Fig. 1a, using the normalization parameters that the state of the art basecaller **Dorado** from ONT provides. They can be found in the **bam** output after basecalling the with **Dorado**. The shift parameter is found under the **sm** tag (scaling midpoint) and the scale parameter is found under the **sd** tag (scaling dispersion). In case of the r9 pore, the pA signal is normalized. For the new rp4 and r10 pores, the raw integer values are normalized directly, skipping the conversion to the pA signal.

$$T_{\text{pA}} = \frac{(T_{\text{DACs}} + \text{offset}) * \text{range}}{\text{digitization}} \quad (1)$$

$$T_{\text{norm}} = \frac{T_{\text{pA}} - \text{shift}}{\text{scale}} \quad (2)$$

Filtering

T can contain erroneous measurements caused by sensor errors while sequencing, Fig. 1b. They manifest as sudden peaks or drops within the signal. Which is why we apply a Hampel filter [14]. It removes outliers in time series data, by iterating it with a sliding window approach. Within the window, each data point is compared to the window median. When a data point deviates from the median by more than a defined threshold, typically expressed as a multiple of the median absolute deviation (MAD), it is identified as an outlier. Each outlier is replaced by the window median. The Hampel filter is insensitive to extreme values, making it effective in handling non-Gaussian noise and preserving the integrity of the underlying signal.

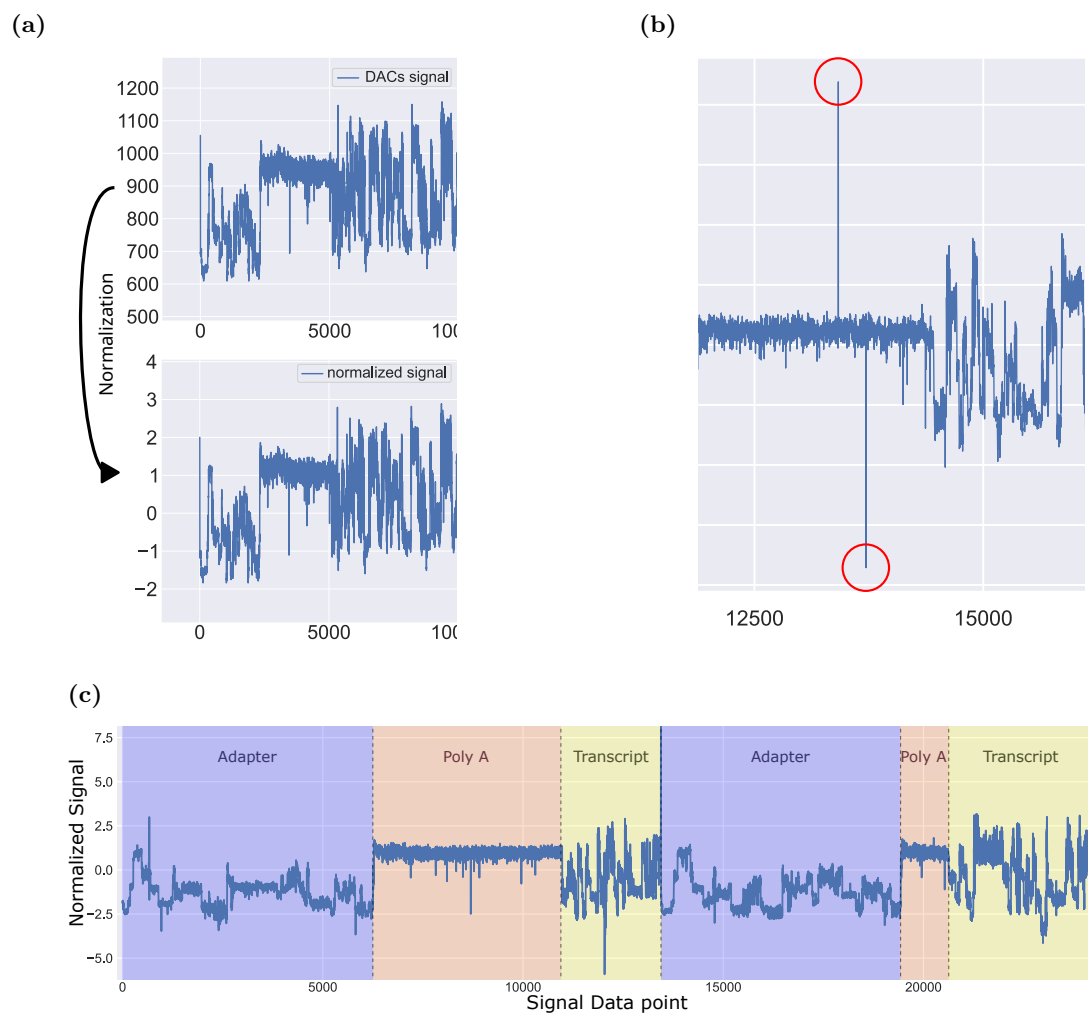


Figure 1: * Overview of the preprocessing steps the signal data. (a) The unnormalized time series is shown at the top, with the corresponding normalized signal at the bottom, highlighting the transformation applied to standardize the data. (b) The time series with detected outliers, indicated by red circles, marking points that deviate significantly from the expected signal pattern mark sensor errors. (c) The entire time series is segmented into three distinct reappearing components in case of RNA sequencing: "adapter," "poly A," and "transcript,". Within a single signal these segments can appear multiple times. *

Trimming

The **bam** file from **Dorado** contains additional tags. The **ts** tag holds the number of trimmed signal data points *ts* from the start of the signal by **Dorado**, e.g. the adapter part in RNA sequencing, Fig. 1c. Tag **ns** is the number of *ts* in the signal prior to trimming. It can happen during library

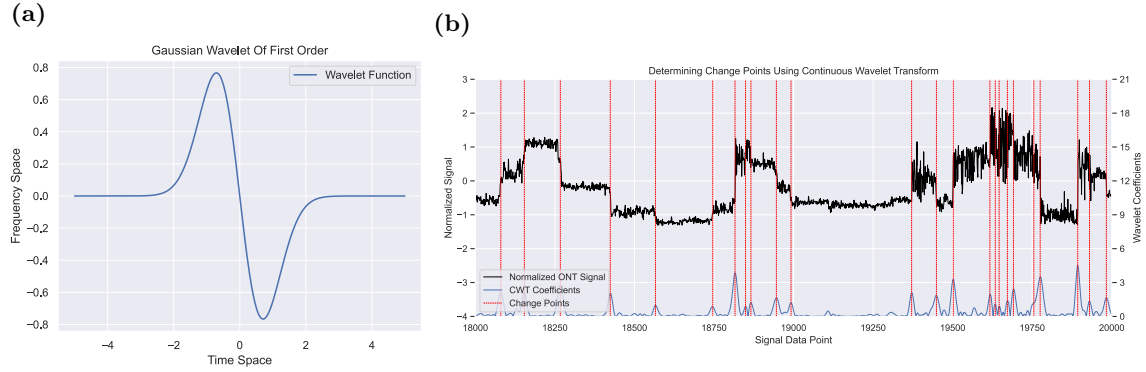


Figure 2: *Visualization of the Gaussian wavelet and its application to the ONT signal. (a) The first-order Gaussian wavelet function ("gaus1") has a sharp and symmetric form, ideal for detecting change points in signals. The x-axis represents time, while the y-axis denotes the amplitude of the signal. (b) An ONT signal with the CWT coefficients. The x-axis corresponds to measured data points (also representing time) and the y-axis displays the normalized signal intensity. The red vertical lines mark the extracted change points, identified as local maxima in the CWT exceeding a threshold of 0.8.*

preparation, that two reads get ligated together, Fig. 1c. These signals will be split by **Dorado**. Split reads get assigned with a new read ID, different from the one in the raw data. They have additional tags, e.g. the **pi** tag, containing the original ID and the **sp** tag marking the split position in T . The normalized and trimmed signal T_{trimmed} , Equ. 3, and read N is passed to **Dynamont**. For the sake of simplicity, the trimmed signal named T with the length $\mathcal{T}$ in the following text.

$$T_{\text{trimmed}} = (t_{sp+ts}, \dots, t_{sp+ns-1}) \quad (3)$$

*

Determine Change Points using Wavelet Transform

* The Continuous Wavelet Transform (CWT) is widely used for pattern recognition in time series data [15–17]. Its temporal localization is particularly effective for analyzing non-stationary signals, such as those from ONT. It decomposes signals into time-frequency coefficients, where the frequency reflects the similarity of the signal and the wavelet function. The frequency is calculated as the inner product of the two. This highlights patterns where the wavelet shape aligns with the signal. The "gaus1" wavelet, which is well suited for detecting sharp changes, identifies sudden shifts during nucleotide transitions in nanopore signals, see Fig. 2a. Using a scale of 6, which matches the longest minimum segment length of the tools in this study (f5c), local maxima of the CWT were extracted. A threshold of 0.8 was applied to the resulting frequency coefficients to ensure that there are not more than $\mathcal{N}$ change points per read, see Fig. S4a and S4b. $\mathcal{N}$ sets a rough upper bound for the number of expected change points in the ONT signal.

*

Dynamont HMM models

better output of dynamont: probability output of the HMM for each segment

* **Dynamont** utilizes HMMs to perform the nucleotide N to signal T alignment. Traditionally, a HMM is used to model sequential systems where the observations (in this case, the sequenced signal T) is explained with hidden states (here a set of overlapping k -mers). These k -mers are probabilistically aligned to the signal, and the transitions describe the likelihood of moving between k -mer states. Emission probabilities capture the likelihood of observing a particular signal value given the current k -mer state. In **Dynamont**, an additional input vector, the nucleotide sequence N base called by **Dorado**, is incorporated into the model. This extends the conventional HMM by aligning both the signal T and the observed nucleotides N . The hidden states represent not only the k -mers that generated the signal but also the alignment context between the signal, the k -mers, and the input nucleotides, Fig. 3 and Equ. 6-7. **Dynamont** implements two models, one performing only the alignment, and another additionally correcting mismatches in the input sequence while performing the alignment. *

Emission

* Nanopore signals are influenced by multiple factors, including nucleotide context and experimental noise, resulting in approximately Gaussian-shaped distributions. The use of the Gaussian probability density function (PDF) ϕ , as defined in Equ. 4, provides a mathematically robust way to capture this variability while maintaining computational efficiency.

$$\phi(t, \mu_k, \sigma_k) = \frac{1}{\sqrt{2\pi\sigma_k^2}} \exp\left(-\frac{(t - \mu_k)^2}{2\sigma_k^2}\right) \quad (4)$$

For each k -mer in the nucleotide sequence N , the parameters μ_k (mean current) and σ_k (standard deviation) are provided by ONT as part of their k -mer model datasets, which are publicly available on their GitHub repository¹. These parameters are derived from experimental data and characterize the expected signal for each k -mer under specific sequencing conditions.

The Gaussian PDF calculates the probability of observing a signal value t given a k -mer, enabling the HMM to quantify how well a particular k -mer explains the observed signal. In the HMMs presented later, ϕ is used uniformly across all states to compute emission probabilities. This approach ensures that the model captures the probabilistic relationship between the observed signal T and the underlying k -mers in N . Additionally, for the NTC HMM, this method supports error correction by leveraging the Gaussian scores to realign mismatched signal segments with the correct k -mers. *

Dynamont NT HMM

* The NT HMM of **Dynamont** aligns N to T and consists of the states A (align) and E (extend), Fig. 3a. A moves (M) in both dimensions N and T , marking the start of a new segment. E moves in T and halts (H) in N , extending a segment, Equ. 5-Equ. 6. Between the states, the transitions $a1$, $e1$, and $e2$ exist. $a1$ starts a new segment. $e1$ is the first extension of a segment and $e2$ further extends the segment. This model architecture limits the minimum segment size to 2.

¹https://github.com/nanoporetech/kmer_models

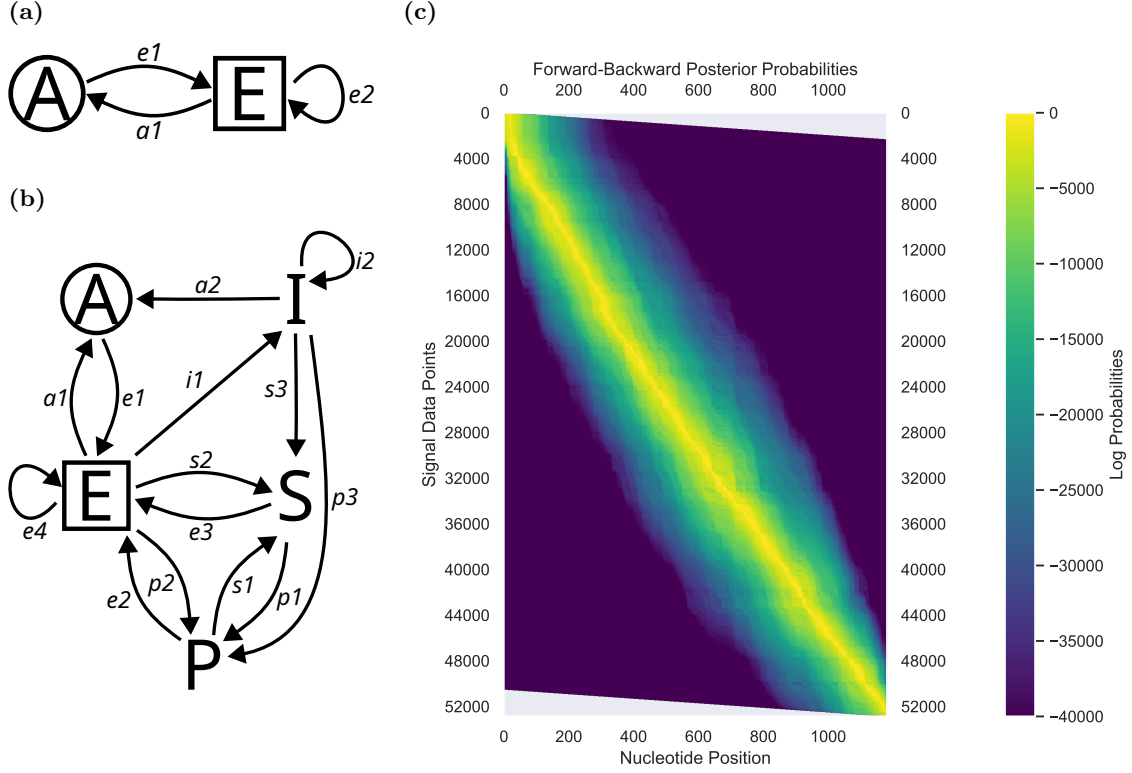


Figure 3: *Letters denote states, circled states mark starting points and squared states end points. Arrows denote transitions between the states. (a) The NT HMM, aligning nucleotide sequence N to time series T . (b) The NTC HMM aligns N to T while correcting errors using C as a third dimension. (c) After Forward-Backward calculations, posterior decoding of the matrices results in the optimal alignment. The possible Maximum A-Posteriori (MAP) is found in a flux of high probability through the $N \times T$ space.*

The Forward Algorithm α of NT HMM iterates N and T from t_0 and k_0 to t_{T-1} and k_{N-k} . For each iteration, α calculates a score using the paths that could be taken to end up in the current state, Equ. 5 [18, 19]. Within the dynamic programming implementation, a matrix exists for each state. The position i, j in each matrix stores the alignment score for the alignment of $(n_0, \dots, n_{j+k-1})$ and $(t_0, \dots, t_i)$. The emission to match k and t is denoted by $\begin{bmatrix} k_j \\ t_i \end{bmatrix}$. The initial position is $M_{0,0}$, denoted by $\begin{bmatrix} \epsilon_{j=0} \\ \epsilon_{i=0} \end{bmatrix}$. The final alignment score Z can be found in $E_{N-k, T-1}$.

$$\begin{aligned} A: \begin{pmatrix} M_j \\ M_i \end{pmatrix} &\rightarrow a1 \begin{pmatrix} M_{j-1} \\ H_{i-1} \end{pmatrix} \begin{bmatrix} k_j \\ t_i \end{bmatrix} \Bigg| \begin{bmatrix} \epsilon_{j=0} \\ \epsilon_{i=0} \end{bmatrix} \\ E: \begin{pmatrix} H_j \\ M_i \end{pmatrix} &\rightarrow e1 \begin{pmatrix} M_j \\ M_{i-1} \end{pmatrix} \begin{bmatrix} k_j \\ t_i \end{bmatrix} \Bigg| e2 \begin{pmatrix} H_j \\ M_{i-1} \end{pmatrix} \begin{bmatrix} k_j \\ t_i \end{bmatrix} \end{aligned} \quad (5)$$

The Backward algorithm β of NT HMM is derived from the Forward algorithm. It is the second pass over the data, iterating from $t_{\mathcal{T}-1}$ and $k_{\mathcal{N}-k}$ toward t_0 and k_0 [18, 19]. The Backward algorithm calculates a score using the paths that could be taken next starting from the current state. The initial position is denoted by $\begin{bmatrix} \delta_{j=\mathcal{N}-k}^* \\ \delta_{i=\mathcal{T}-1}^* \end{bmatrix}$ in position $E_{\mathcal{N}-k, \mathcal{T}-1}$. The final alignment score Z^* can be found in $M_{0,0}$.

$$\begin{aligned} A^* : \begin{pmatrix} M_j \\ M_i \end{pmatrix} &\rightarrow e1 \begin{pmatrix} H_j \\ M_{i+1} \end{pmatrix} \begin{bmatrix} k_j \\ t_{i+1} \end{bmatrix} \\ E^* : \begin{pmatrix} H_j \\ M_i \end{pmatrix} &\rightarrow a1 \begin{pmatrix} M_{i+1} \\ M_{i+1} \end{pmatrix} \begin{bmatrix} k_{j+1} \\ t_{i+1} \end{bmatrix} \mid e2 \begin{pmatrix} H_j \\ M_{i+1} \end{pmatrix} \begin{bmatrix} k_j \\ t_{i+1} \end{bmatrix} \mid \begin{bmatrix} \delta_{j=\mathcal{N}-k}^* \\ \delta_{i=\mathcal{T}-1}^* \end{bmatrix} \end{aligned} \quad (6)$$

Within the Forward and Backward algorithm of NT HMM, N and T are fully iterated, giving the NT HMM a runtime complexity of $O(\mathcal{NT})$. For each state, a matrix of size $\mathcal{N}$ times $\mathcal{T}$ has to be stored, leading to a memory complexity of $O(\mathcal{NT})$. *

NTC HMM

* NTC HMM aligns N to T while doing an error correction using C , Fig. 3b. It consists of 5 states: A (align), E (extend), I (insertion), S (sequence), and P (polish). A and E are similar to NT HMM. A moves in all dimensions, aligning t_i , k_j , and c_h . c_h is a k -mer of C with the index h and k_j is the k -mer of N at position j . c_h and k_j may differ. A starts a new segment for k_j and c_h at the same "time" t_i . E moves in T , while halting in N and C , extending the current segment. I handles the insertion of nucleotides in N , by moving only in N , while halting in T and C . P does the opposite, by moving in T and C it can handle deletions and additional polishings. S moves in N and T , while halting in C . While A allows a simultaneous segment start in all dimensions, P and S allow for different segment starts end ends of the N to T alignment versus the T to C alignment.

The Forward Algorithm α of NTC HMM iterates N and T similar to NT HMM but the iteration of C is different. To move from one c to the next, $p(h)$ is introduced. This function returns the indices of preceding k -mers of c_h in C . For example, if $c_h = \text{AGGTC}$ then $c_{p(h)}$ are AAGGT, CAGGT, GAGGT, and GAGGT, if $\Sigma = \{A, C, G, T\}$. All rules using $p(h)$ are called $|\Sigma|$ times.

$$\begin{aligned} A : \begin{pmatrix} M_j \\ M_i \\ M_h \end{pmatrix} &\rightarrow a1 \begin{pmatrix} H_{j-1} \\ M_{i-1} \\ H_{p(h)} \end{pmatrix} \begin{bmatrix} k_j \\ t_i \\ c_{h_h} \end{bmatrix} \mid a2 \begin{pmatrix} M_{j-1} \\ H_{i-1} \\ H_{p(h)} \end{pmatrix} \begin{bmatrix} k_j \\ t_i \\ c_{h_h} \end{bmatrix} \mid \begin{bmatrix} \epsilon_j=0 \\ \epsilon_i=0 \\ c_h \in C \end{bmatrix} \\ I : \begin{pmatrix} M_j \\ H_i \\ H_h \end{pmatrix} &\rightarrow i1 \begin{pmatrix} H_{j-1} \\ M_i \\ H_h \end{pmatrix} \begin{bmatrix} k_j \\ t_i \\ c_{h_h} \end{bmatrix} \mid i2 \begin{pmatrix} M_{j-1} \\ H_i \\ H_h \end{pmatrix} \begin{bmatrix} k_j \\ t_i \\ c_{h_h} \end{bmatrix} \\ P : \begin{pmatrix} H_j \\ M_i \\ M_h \end{pmatrix} &\rightarrow p1 \begin{pmatrix} M_j \\ M_{i-1} \\ H_{p(h)} \end{pmatrix} \begin{bmatrix} k_j \\ t_i \\ c_{h_h} \end{bmatrix} \mid p2 \begin{pmatrix} H_j \\ M_{i-1} \\ H_{p(h)} \end{pmatrix} \begin{bmatrix} k_j \\ t_i \\ c_{h_h} \end{bmatrix} \mid p3 \begin{pmatrix} M_j \\ H_{i-1} \\ H_{p(h)} \end{pmatrix} \begin{bmatrix} k_j \\ t_i \\ c_{h_h} \end{bmatrix} \\ S : \begin{pmatrix} M_j \\ M_i \\ H_h \end{pmatrix} &\rightarrow s1 \begin{pmatrix} H_{j-1} \\ M_{i-1} \\ M_h \end{pmatrix} \begin{bmatrix} k_j \\ t_i \\ c_{h_h} \end{bmatrix} \mid s2 \begin{pmatrix} H_{j-1} \\ M_{i-1} \\ H_h \end{pmatrix} \begin{bmatrix} k_j \\ t_i \\ c_{h_h} \end{bmatrix} \mid s3 \begin{pmatrix} M_{j-1} \\ H_{i-1} \\ H_h \end{pmatrix} \begin{bmatrix} k_j \\ t_i \\ c_{h_h} \end{bmatrix} \\ E : \begin{pmatrix} H_j \\ M_i \\ H_h \end{pmatrix} &\rightarrow e1 \begin{pmatrix} M_j \\ M_{i-1} \\ M_h \end{pmatrix} \begin{bmatrix} k_j \\ t_i \\ c_{h_h} \end{bmatrix} \mid e2 \begin{pmatrix} H_j \\ M_{i-1} \\ M_h \end{pmatrix} \begin{bmatrix} k_j \\ t_i \\ c_{h_h} \end{bmatrix} \mid e3 \begin{pmatrix} M_j \\ M_{i-1} \\ H_h \end{pmatrix} \begin{bmatrix} k_j \\ t_i \\ c_{h_h} \end{bmatrix} \mid e4 \begin{pmatrix} H_j \\ M_{i-1} \\ H_h \end{pmatrix} \begin{bmatrix} k_j \\ t_i \\ c_{h_h} \end{bmatrix} \end{aligned} \quad (7)$$

As the alignment is not limited in C , the final alignment score Z is not found at a specific index, but as a sum over the last column:

$$Z = \sum_{c_h \in C} E_{N-k, \mathcal{T}-1, h}$$

The Backward Algorithm β of NTC HMM iterates, backwards over the indices. Here $s(h)$ is the inverse function of $p(h)$. $s(h)$ returns the indices in C of the succeeding k -mers of c_h , e.g. if $c_h = \text{AGGTC}$, the succeeding k -mers are GGTCA , GGTCC , GGTCG , and GGTCT .

$$\begin{aligned}
A^* : \begin{pmatrix} M_j \\ M_i \\ M_h \end{pmatrix} &\rightarrow e1 \begin{pmatrix} H_j \\ M_{i+1} \\ H_h \end{pmatrix} \begin{bmatrix} k_j \\ t_{i+1} \\ c_h \end{bmatrix} \\
I^* : \begin{pmatrix} M_j \\ H_i \\ H_h \end{pmatrix} &\rightarrow a2 \begin{pmatrix} M_{j+1} \\ M_{i+1} \\ M_{s(h)} \end{pmatrix} \begin{bmatrix} k_{j+1} \\ t_{i+1} \\ c_{s(h)} \end{bmatrix} \left| \begin{array}{l} i2 \begin{pmatrix} M_{j+1} \\ H_i \\ H_h \end{pmatrix} \begin{bmatrix} k_{j+1} \\ t_i \\ c_h \end{bmatrix} \\ p3 \begin{pmatrix} H_j \\ M_{i+1} \\ M_{s(h)} \end{pmatrix} \begin{bmatrix} k_j \\ t_{i+1} \\ c_{s(h)} \end{bmatrix} \right| s3 \begin{pmatrix} M_{j+1} \\ M_{i+1} \\ H_h \end{pmatrix} \begin{bmatrix} k_{j+1} \\ t_{i+1} \\ c_h \end{bmatrix} \\
P^* : \begin{pmatrix} H_j \\ M_i \\ M_h \end{pmatrix} &\rightarrow s1 \begin{pmatrix} M_{j+1} \\ M_{i+1} \\ H_h \end{pmatrix} \begin{bmatrix} k_{j+1} \\ t_{i+1} \\ c_h \end{bmatrix} \left| e2 \begin{pmatrix} H_j \\ M_{i+1} \\ H_h \end{pmatrix} \begin{bmatrix} k_j \\ t_{i+1} \\ c_h \end{bmatrix} \right. \\
S^* : \begin{pmatrix} M_j \\ M_i \\ H_h \end{pmatrix} &\rightarrow p1 \begin{pmatrix} H_j \\ M_{i+1} \\ M_{s(h)} \end{pmatrix} \begin{bmatrix} k_j \\ t_{i+1} \\ c_{s(h)} \end{bmatrix} \left| e3 \begin{pmatrix} H_j \\ M_{i+1} \\ H_h \end{pmatrix} \begin{bmatrix} k_j \\ t_{i+1} \\ c_h \end{bmatrix} \right. \\
E^* : \begin{pmatrix} H_j \\ M_i \\ M_h \end{pmatrix} &\rightarrow a1 \begin{pmatrix} M_{j+1} \\ M_{i+1} \\ M_{s(h)} \end{pmatrix} \begin{bmatrix} k_{j+1} \\ t_{i+1} \\ c_{s(h)} \end{bmatrix} \left| i1 \begin{pmatrix} M_{j+1} \\ H_i \\ H_h \end{pmatrix} \begin{bmatrix} k_{j+1} \\ t_i \\ c_h \end{bmatrix} \right| p2 \begin{pmatrix} H_j \\ M_{i+1} \\ M_{s(h)} \end{pmatrix} \begin{bmatrix} k_j \\ t_{i+1} \\ c_{s(h)} \end{bmatrix} \left| \begin{array}{l} s2 \begin{pmatrix} M_{j+1} \\ M_{i+1} \\ H_h \end{pmatrix} \begin{bmatrix} k_{j+1} \\ t_{i+1} \\ c_h \end{bmatrix} \\ e4 \begin{pmatrix} H_j \\ M_{i+1} \\ H_h \end{pmatrix} \begin{bmatrix} k_j \\ t_{i+1} \\ c_h \end{bmatrix} \right| \begin{bmatrix} \delta_{j=\mathcal{N}-k}^* \\ \delta_{i=\mathcal{T}-1}^* \\ c_h \in C \end{bmatrix}
\end{aligned} \tag{8}$$

The alignment score Z^* is calculated by:

$$Z^* = \sum_{c_h \in C} A_{0,0,h}$$

Analogous to NT HMM the runtime and memory complexity for NTC HMM is $O(\mathcal{NT}|C|)$. *

Alignment Extraction using MAP path

* Posterior probabilities P_Q are calculated for each Q and Q^* . Z is used to standardize the probabilities, Equ. 9.

$$P_{Q_{i,j}} = \frac{Q_{i,j} Q_{i,j}^*}{Z} \tag{9}$$

The MAP path is extracted using $P_{Q_{i,j}}$ with the Viterbi algorithm [19]. It corresponds to the optimal solution of the N to T alignment of the given model, Fig. 3c. *

Tombo Resquiggle

* Tombo Resquiggle² (v1.5.1) aligns ONT signals and a reference sequence using a mapping, signal normalization, event detection, and a sequence to signal alignment. It takes the `single fast5`

²<https://nanoporetech.github.io/tombo/resquiggle.html>

format (an older fast5 format), a **fastq** and a reference sequence **fasta** as input. After aligning the reads and the reference sequence, the raw signal is normalized for per-read consistency. A signal is normalized using the median shift and MAD scaling to standardize signal levels. Dynamic event detection segments the signal into meaningful chunks, which are then aligned with the reference using a banded dynamic programming approach. Tombo ensures robust sequence-to-signal assignments, critical for accurate downstream analyses such as modified base detection or visualization. *

f5c

* **f5c** (v1.5) [10] is an optimized re implementation of **nanopolish**³ (v0.14) [11]. **f5c eventalign** aligns ONT signals with a reference sequence. It uses a HMM framework to align the unnormalized signals to the expected current levels of reference k -mers. The alignment leverages a pore model, which describes the expected mean and variance of the pA current for each k -mer, similar to **dynamont**. **f5c** takes base called reads in **fastq** format, a reference genome in **fasta**, and a pre-aligned sorted **bam** mapping file. Another tool, **f5c resquiggle**, does the same without using a pre-aligned **bam** file. It aligns the unnormalized pA signal and the base called reads directly. *

dorado

* **Dorado**⁴ (v0.8.3) is the state of the art basecaller by ONT. It utilizes a neural network and a window approach using a step size of 5 data points to convert the sequencing signal into nucleotide sequences. Therefore, it provides a low resolution segmentation for every 5th data point. **Dorado** takes the raw signal in **pod5** or **fast5** format as input and outputs the base calls together with additional information in **bam** format. *

Results & Discussion

* Achieving high-quality signal segmentation for ONT signals is essential for high-resolution downstream analyses, such as detecting and studying RNA modifications. Precise segmentation is particularly important because RNA modifications introduce unknown signal changes that lack Gaussian models, making it challenging to analyze their effects without accurately partitioning the signal into meaningful regions. A well-segmented signal allows for the detailed examination of unique signal distributions, which is critical to study modifications.

For each ground truth change point determined by the CWT approach, tool predictions were evaluated within a defined distance threshold. It is important to note that this comparison is qualitative rather than quantitative, as the change points identified by the CWT method were not experimentally validated. A prediction was considered valid only if it matched a change point within the threshold and was closer to that change point than any other, ensuring that each prediction was matched to a single closest change point.

We evaluated **Dynamont**, **Dorado**, **f5c**, and **Tombo**, along with positive controls, using 10,000 random reads from the HEK293T (RNA002 or R9 pore) and UHR (RNA004 or RP4 pore) datasets. Random controls assigned $\mathcal{N}$ (number of bases) segmentation borders randomly within the signal

³<https://simpsonlab.github.io/2015/04/08/eventalign/>

⁴<https://github.com/nanoporetech/dorado>

T for each read, while uniform controls placed $\mathcal{N}$ equidistant segmentation borders. All tools were executed with disabled quality filters, see Sec. S1.4 – S1.6. *

RNA r9

*

- Dynamont NT, NTC and f5c resquiggle dominate
- NTC works a little bit better, but needs a lot more time and memory
- adding positive found and negative found is greater than 1.0 because 0 is counted twice
- dist threshold negative means: found with distance 0 or greater than dist
- dist threshold positive means: found with distance 0 or smaller than dist
- more change points found after the ground truth change point from the wavelet ($gt - pred < 0 \iff pred > gt$)
- Dynamont has a trend to put the segment border before the ground truth
- f5c is uniformly distributed
- Tombo has a strong trend to set border after
- Dorado
- Controls also show a trend to be after ground truth change point
- dynamont segments a lot more reads compared to the other tools

** Out of the random 10,000 reads of the HEK293T dataset, 9,999 were successfully basecalled and used for tool comparisons. At an absolute distance of 0 between the predicted segmentation border and the ground truth change point, **Dynamont NTC**, **Dynamont NT**, and **f5c Resquiggle** perform similarly, see Fig.4a and Tab.1. However, as the accepted distance threshold increases, **Dynamont NTC** and **Dynamont NT** consistently outperform the other tools, showcasing superior accuracy. *

RNA rp4

Conclusion

References

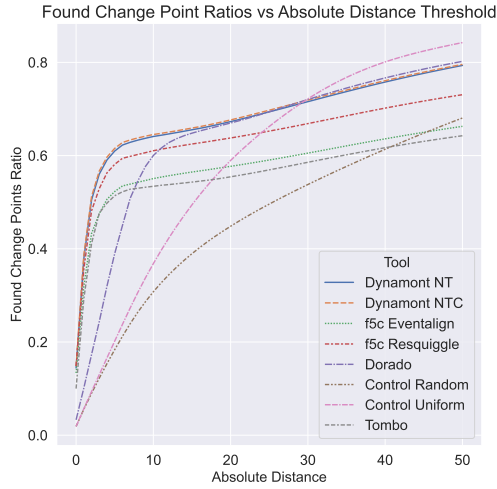
- [1] Miten Jain et al. “The Oxford Nanopore MinION: delivery of nanopore sequencing to the genomics community”. In: *Genome Biology* 17.1 (2016).
- [2] Yunhao Wang et al. “Nanopore sequencing technology, bioinformatics and applications”. In: *Nature Biotechnology* 39.11 (2021), pp. 1348–1365.
- [3] Daniel R Garalde et al. “Highly parallel direct RNA sequencing on an array of nanopores”. In: *Nature Methods* 15.3 (2018), pp. 201–206.

Table 1: Detection of Change Points at Various Distance Thresholds

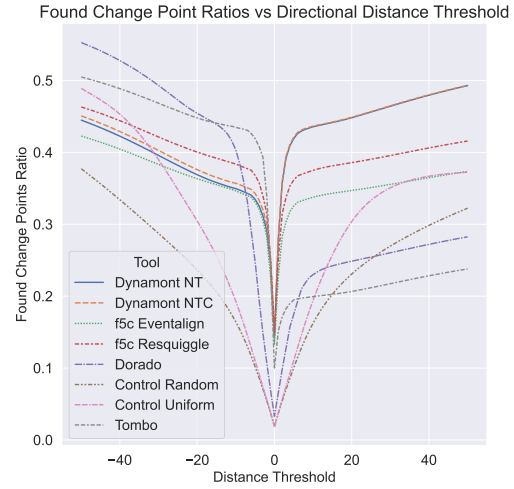
Dist.	Dyn. NT	Dyn. NTC	f5c Res.	f5c Event.	Tombo	Dorado
0	1,189,976 (14.4%)	1,223,437 (14.8%)	1,223,843 (14.8%)	1,100,728 (13.3%)	825,563 (10.0%)	272,541 (3.3%)
1	3,127,281 (37.9%)	3,192,367 (38.7%)	2,938,322 (35.6%)	2,640,586 (32.0%)	2,414,284 (29.3%)	834,890 (10.1%)
2	4,165,470 (50.5%)	4,224,630 (51.2%)	3,973,563 (48.1%)	3,571,442 (43.3%)	3,393,760 (41.1%)	1,427,170 (17.3%)
3	4,623,802 (56.0%)	4,678,214 (56.7%)	4,366,812 (52.9%)	3,924,634 (47.6%)	3,931,462 (47.6%)	2,010,977 (24.4%)
5	5,032,509 (60.9%)	5,079,959 (61.6%)	4,797,324 (58.1%)	4,315,415 (52.3%)	4,241,159 (51.4%)	3,217,436 (38.9%)
10	5,290,327 (64.1%)	5,325,388 (64.5%)	5,037,920 (61.0%)	4,544,753 (55.1%)	4,407,926 (53.4%)	4,953,890 (60.0%)

- [4] Jared T Simpson et al. “Detecting DNA cytosine methylation using nanopore sequencing”. In: *Nature Methods* 14.4 (2017), pp. 407–410.
- [5] Xichen Zhao et al. “Detecting RNA modification using direct RNA sequencing: A systematic review”. In: *Computational and Structural Biotechnology Journal* 20 (2022), pp. 5740–5749.
- [6] Daniel Branton et al. “The potential and challenges of nanopore sequencing”. In: *Nature Biotechnology* 26.10 (2008), pp. 1146–1153.
- [7] Gerald M Cherf et al. “Automated forward and reverse ratcheting of DNA in a nanopore at 5-Å precision”. In: *Nature Biotechnology* 30.4 (2012), pp. 344–348.
- [8] David Deamer et al. “Three decades of nanopore sequencing”. In: *Nature Biotechnology* 34.5 (2016), pp. 518–524.
- [9] Franka J. Rang et al. “From squiggle to basepair: computational approaches for improving nanopore sequencing read accuracy”. In: *Genome Biology* 19.1 (2018).
- [10] Hasindu Gamaarachchi et al. “GPU accelerated adaptive banded event alignment for rapid comparative nanopore signal analysis”. In: *BMC bioinformatics* 21.1 (2020), pp. 1–13.
- [11] Nicholas J Loman et al. “A complete bacterial genome assembled de novo using only nanopore sequencing data”. In: *Nature Methods* 12.8 (2015), pp. 733–735.
- [12] Ploy N. Pratanwanich et al. “Identification of differential RNA modifications from nanopore direct RNA sequencing with xPore”. In: *Nature Biotechnology* 39.11 (2021), pp. 1394–1402.
- [13] Hiruna Samarakoon et al. “Flexible and efficient handling of nanopore sequencing signal data with slow5tools”. In: *Genome Biology* 24.1 (2023).
- [14] Ronald K. Pearson et al. “Generalized Hampel Filters”. In: *EURASIP Journal on Advances in Signal Processing* 2016.1 (2016).
- [15] Ali Soleymani et al. “Characterizing change points and continuous transitions in movement behaviours using wavelet decomposition”. In: *Methods in Ecology and Evolution* 8.9 (2017). Ed. by Oscar Gaggiotti, pp. 1113–1123.

(a)



(b)



(c)

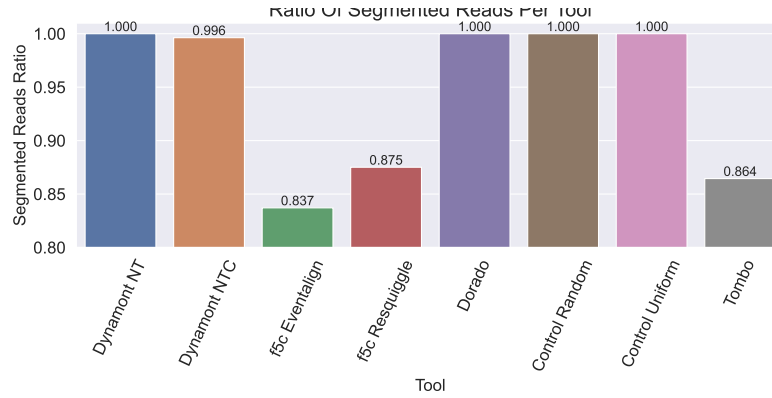


Figure 4: *Analysis of change point detection and read segmentation across tools. (a) Cumulative ratio of found change points versus the Absolute Distance Threshold. The x-axis ranges from 0 to 50, with the y-axis representing the proportion of detected change points within the respective thresholds. (b) Cumulative ratio of found change points versus the Distance Threshold. Negative thresholds reflect cumulative ratios of 0 for distances below the threshold, while positive thresholds show cumulative ratios for distances up to the threshold. (c) Proportion of successfully segmented reads per tool. This highlights cases where tools either filtered out reads or failed to provide segmentations for certain reads.*

- [16] G.K. Rajini. “A comprehensive review on Wavelet Transform and its applications”. In: 11 (2016), pp. 11713–11723.
- [17] Luís Aguiar-Conraria et al. “THE CONTINUOUS WAVELET TRANSFORM: MOVING BEYOND UNI- AND BIVARIATE ANALYSIS”. In: *Journal of Economic Surveys* 28.2 (2013), pp. 344–375.
- [18] L.R. Rabiner. “A tutorial on hidden Markov models and selected applications in speech recognition”. In: *Proceedings of the IEEE* 77.2 (1989), pp. 257–286.
- [19] Richard Durbin et al. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press, 1998, pp. 54–66.
- [20] Mark Gales et al. “The Application of Hidden Markov Models in Speech Recognition”. In: *Foundations and Trends® in Signal Processing* 1.3 (2007), pp. 195–304.
- [21] Bing-Hwang Juang et al. “Maximum likelihood estimation for multivariate mixture observations of markov chains (Corresp.)” In: *IEEE Transactions on Information Theory* 32.2 (1986), pp. 307–309.

Supplements

S1 Commands

S1.1 Benchmark Preparation For RNA r9: HEK293T

```

1  ## Downloads
2  wget ftp://ftp.sra.ebi.ac.uk/vol1/run/ERR470/ERR4706156/HEK293T-WT-rep1.tar.gz
3  wget https://ftp.ensembl.org/pub/release-113/fasta/homo_sapiens/cdna/Homo_sapiens.
   GRCh38.cdna.all.fa
4  ## Data Preparation
5  tar xvf HEK293T-WT-rep1.tar.gz
6  pod5 convert fast5 HEK293T-WT-rep1/*.fast5 -o HEK293T-WT-rep1.pod5
7  ## Extract 10.000 Random Reads
8  pod5 view HEK293T-WT-rep1.pod5 --ids --no-header -o all_ids.txt
9  sort --random-sort all_ids.txt | head --lines 10000 > 10k_ids.txt
10 pod5 filter HEK293T-WT-rep1.pod5 -o HEK293T-WT-rep1_r10k.pod5 --ids 10k_ids.txt
11 ## Convert To Other Data Formats
12 blue-crab p2s HEK293T-WT-rep1_r10k.pod5 -o HEK293T-WT-rep1_r10k.blow5
13 pod5 convert to_fast5 HEK293T-WT-rep1_r10k.pod5 -o fast5/
14 multi_to_single_fast5 -i fast5/ -s single_fast5/
15 ## Basecalling
16 dorado basecaller -x cuda:0 rna002_70bps_hac@v3 HEK293T-WT-rep1_r10k.pod5 > HEK293T-
   WT-rep1_r10k.bam
17 dorado basecaller --emit-fastq -x cuda:0 rna002_70bps_hac@v3 HEK293T-WT-rep1_r10k.
   pod5 > HEK293T-WT-rep1_r10k.fastq
18 ## Mapping
19 minimap2 Homo_sapiens.GRCh38.cdna.all.fa HEK293T-WT-rep1_r10k.fastq -x splice -a |
   samtools view -hbF4 | samtools sort > HEK293T-WT-rep1_r10k_mapping.bam
20 samtools index HEK293T-WT-rep1_r10k_mapping.bam

```

S1.2 Benchmark Preparation For RNA rp4: UHR

```

1  ## Downloads
2  wget https://gtgseq.s3.amazonaws.com/ont-rna004-rna/UHR/raw/PNXRXX240011_reads.blow5
3  wget https://ftp.ensembl.org/pub/release-113/fasta/homo_sapiens/cdna/Homo_sapiens.
   GRCh38.cdna.all.fa
4  ## Data Preparation
5  blue-crab s2p PNXRXX240011_reads.blow5 -o PNXRXX240011.pod5
6  ## Extract 10.000 Random Reads
7  pod5 view PNXRXX240011.pod5 --ids --no-header -o all_ids.txt
8  sort --random-sort all_ids.txt | head --lines 10000 > 10k_ids.txt
9  pod5 filter PNXRXX240011.pod5 -o PNXRXX240011_r10k.pod5 --ids 10k_ids.txt
10 ## Convert To Other Data Formats
11 blue-crab p2s PNXRXX240011_r10k.pod5 -o PNXRXX240011_r10k.blow5
12 ## Basecalling
13 dorado basecaller -x cuda:0 rna004_130bps_sup@v5.1.0 PNXRXX240011_r10k.pod5 >
   PNXRXX240011_r10k.bam
14 dorado basecaller --emit-fastq -x cuda:0 rna004_130bps_sup@v5.1.0 PNXRXX240011_r10k.
   pod5 > PNXRXX240011_r10k.fastq
15 ## Mapping
16 minimap2 Homo_sapiens.GRCh38.cdna.all.fa PNXRXX240011_r10k.fastq -x splice -a |
   samtools view -hbF4 | samtools sort > PNXRXX240011_r10k_mapping.bam
17 samtools index PNXRXX240011_r10k_mapping.bam

```

S1.3 Extracting Dorado Segmentation

```

1  # HEK293T
2  ## Basecalling with emit moves
3  dorado basecaller -x cuda:0 --emit-moves rna002_70bps_hac@v3 HEK293T-WT-rep1_r10k.
   pod5 > HEK293T-WT-rep1_r10k_moves.bam
4  ## Extracting moves as segmentation borders
5  python extractDoradoMoves.py HEK293T-WT-rep1_r10k_moves.bam -o HEK293T-WT-
   rep1_r10k_moves.tsv
6
7  # UHR
8  ## Basecalling with emit moves
9  dorado basecaller -x cuda:0 --emit-moves sup PNXRXX240011_r10k.pod5 >
   PNXRXX240011_r10k_moves.bam
10 ## Extracting moves as segmentation borders
11 python extractDoradoMoves.py PNXRXX240011_r10k_moves.bam -o PNXRXX240011_r10k_moves.
   tsv

```

S1.4 Extracting f5c Segmentation

```

1  # HEK293T
2  f5c index --slow5 .HEK293T-WT-rep1_r10k.blow5 HEK293T-WT-rep1_r10k.fastq
3  ## Eventalign
4  f5c eventalign -b HEK293T-WT-rep1_r10k_mapping.bam -g Homo_sapiens.GRCh38.cdna.all.
   fa -r HEK293T-WT-rep1_r10k.fastq --slow5 HEK293T-WT-rep1_r10k.blow5 --rna --
   signal-index --collapse-events --pore r9 --min-mapq 0 --summary HEK293T-WT-
   rep1_r10k_event.sum > HEK293T-WT-rep1_r10k_event.tsv
5  ## Resquiggle
6  f5c resquiggle --rna --pore r9 HEK293T-WT-rep1_r10k.fastq HEK293T-WT-rep1_r10k.blow5
   > HEK293T-WT-rep1_r10k_resqu.tsv
7
8  # UHR

```

```

9 f5c index --slow5 PNXX240011_r10k.blow5 PNXX240011_r10k.fastq
10 ## Eventalign
11 f5c eventalign -b PNXX240011_r10k_mapping.bam -g Homo_sapiens.GRCh38.cdna.all.fa -
    r PNXX240011_r10k.fastq --slow5 PNXX240011_r10k.blow5 --rna --signal-index
    --collapse-events --pore rna004 --min-mapq 0 --summary PNXX240011_r10k_event.
    sum > PNXX240011_r10k_event.tsv
12 ## Resquiggle
13 f5c resquiggle --rna --pore rna004 PNXX240011_r10k.fastq PNXX240011_r10k.blow5 >
    PNXX240011_r10k_resqu.tsv

```

S1.5 Extracting Tombo Segmentation

```

1 # HEK293T
2 tombo preprocess annotate_raw_with_fastqs --fast5-basedir single_fast5/ --fastq-
    filenames HEK293T-WT-rep1_r10k.fastq --sequencing-summary-filenames
    sequencing_summary.txt
3 tombo resquiggle --q-score 0 --rna single_fast5/ Homo_sapiens.GRCh38.cdna.all.fa

```

S1.6 Extracting Dynamont Segmentation

```

1 # HEK293T
2 python segment.py --raw . --basecalls HEK293T-WT-rep1_r10k.bam --mode basic --
    model_path rna_r9.4_180mv_70bps.model -o HEK293T-WT-rep1_r10k_dynamont.csv --
    pore rna_r9
3 # UHR
4 python segment.py --raw . --basecalls PNXX240011_r10k.bam --mode basic --
    model_path rna_r9.4_180mv_70bps.model -o PNXX240011_r10k_dynamont.csv --pore
    rna_rp4

```

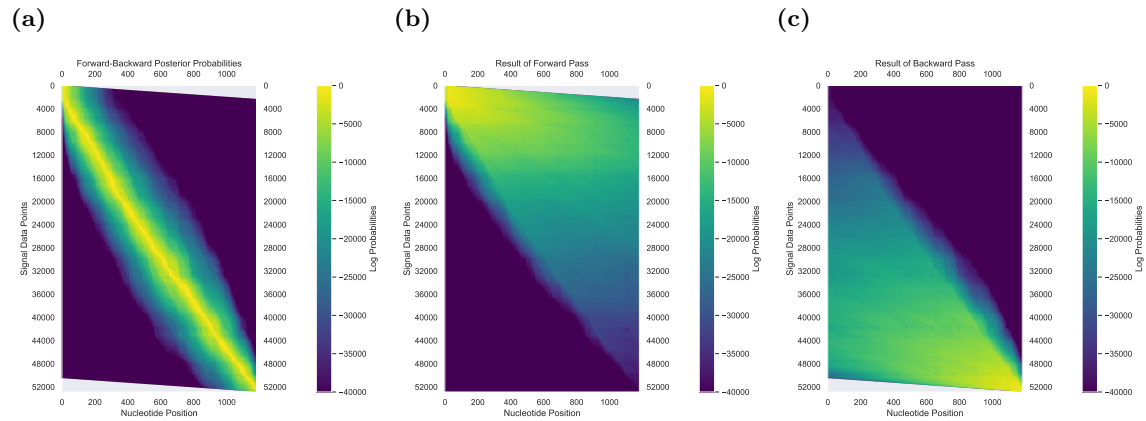


Figure S1: *Full N x T matrices.*

Training Parameters with the Baum-Welch Algorithm

* Using the Forward-Backward Algorithm, we can utilize the Baum-Welch algorithm [18, 19] to train our parameters. It is a special case of the Expectation Maximization Algorithm to find unknown transition and emission parameters of an HMM.

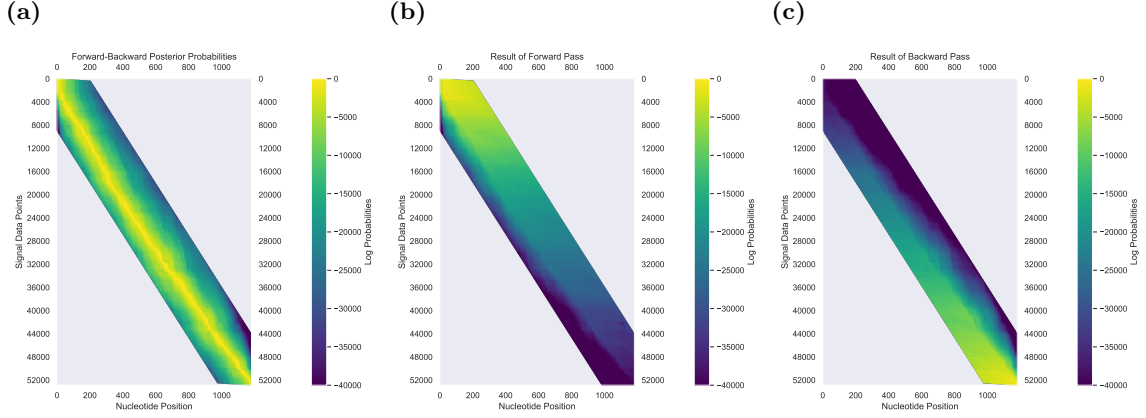


Figure S2: *Fixed banded DP N x T matrices.*

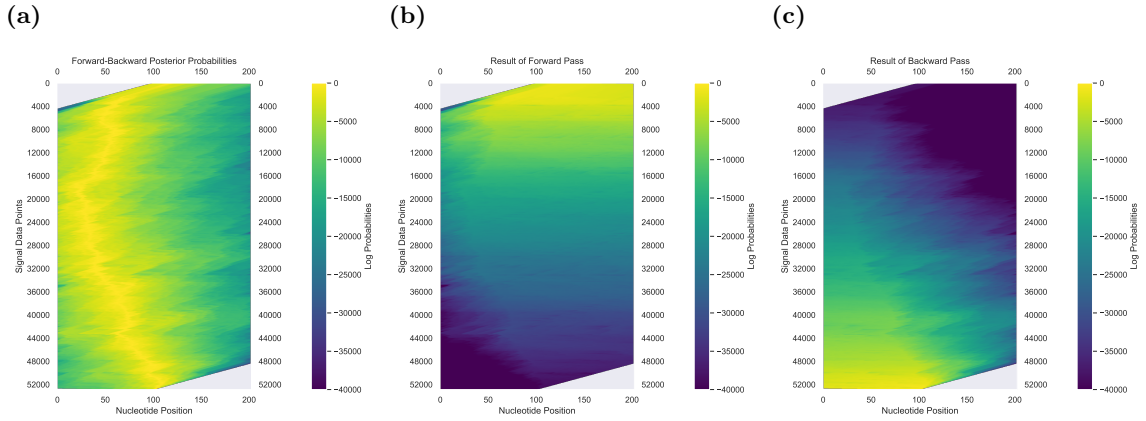


Figure S3: *Fixed banded DP N x T matrices with reduced matrix sizes (only storing the bands)*

Training transition parameters

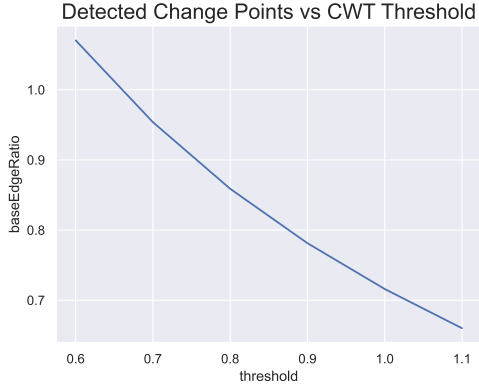
The transition parameters are trained by calculating how many times they were used given a training pair of T and N . Given the states q and r , a_{qr} is the transition probability from q to r . The new transition $\hat{a}_{qr}$ is calculated with Equ. 10.

$$A_{qr} = \frac{\sum_x \alpha_q(x) a_{qr} \phi(o_{x+1}) \beta_r(x+1)}{Z}$$

$$\hat{a}_{qr} = \frac{A_{qr}}{\sum_{r'} A_{qr'}} \quad (10)$$

$\sum_x$ iterates the observations o , in our case k_j and t_i . $\alpha_q(x)$ is the score in state q in α at x . $\beta_r(x+1)$ is the score of state r in β when taking a_{qr} to $x+1$. a_{qr} is the transition probability for the path from $\alpha_q(x)$ to $\beta_r(x+1)$. $\phi(o_{x+1})$ is the emission for observing o_{x+1} . Each transition probability needs to be normalized afterwards to ensure that all outgoing transitions sum up to 1.

(a)



(b)

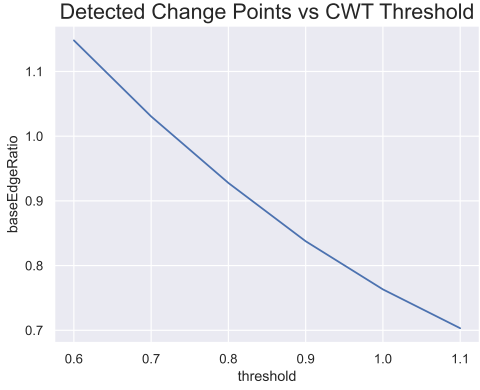


Figure S4: *CWT peak height threshold against number of peaks vs read length.)*

Training emission parameters

New model parameters $\hat{\mu}_k$ and $\hat{\sigma}_k$ for the Gaussian distributions are trained with the Baum-Welch algorithm [20, 21]. First, the expectation step calculates weights for all observations, Equ. 11. For each i and j , the sum of scores of all matrices Q in α get multiplied with the sum of scores of all matrices Q^* in β , which is standardized by Z . γ is a matrix that stores these weights for all i and j .

$$\gamma_{i,j} = \frac{\sum_Q Q_{i,j} \sum_{Q^*} Q_{i,j}^*}{Z} \quad (11)$$

Within the maximization step, Equ. 12, the weights in γ are used to calculate a weighted sum of the observations t_i , which will be standardized with the sum of weights over i of γ , to get $\hat{\mu}_j$. As a k -mer can appear multiple times in N , $\hat{\mu}_k$ is calculated by taking the average $\hat{\mu}_j$ for all $k_j = k$ in N . The same happens for $\hat{\sigma}_k$.

$$\begin{aligned} \hat{\mu}_j &= \frac{\sum_{i=1}^{T-1} \gamma_{i,j} t_i}{\sum_{i=1}^{T-1} \gamma_{i,j}} & \hat{\sigma}_j^2 &= \frac{\sum_{i=1}^{T-1} \gamma_{i,j} (t_i - \hat{\mu}_j)^2}{\sum_{i=1}^{T-1} \gamma_{i,j}} \\ \hat{\mu}_k &= \frac{\sum_{j:k_j=k} \hat{\mu}_j}{\text{counts}_k(N)} & \hat{\sigma}_k &= \sqrt{\frac{\sum_{j:k_j=k} \hat{\sigma}_j^2}{\text{counts}_k(N)}} \end{aligned} \quad (12)$$

*